

Chương 6. Kiến trúc bộ nhớ máy tính

- 6.1. Các cấp bộ nhớ
- 6.2. Khái niệm các loại bộ nhớ
- 6.3. Mức Cache – Bộ nhớ trong
- 6.4. Bộ nhớ ảo
- 6.5. Tổ chức của chip nhớ
- 6.6. Phát hiện và chỉnh lỗi trong bộ nhớ

6.1. Các cấp bộ nhớ

Dung lượng
Thời gian truy cập
Giá tiền / đơn vị

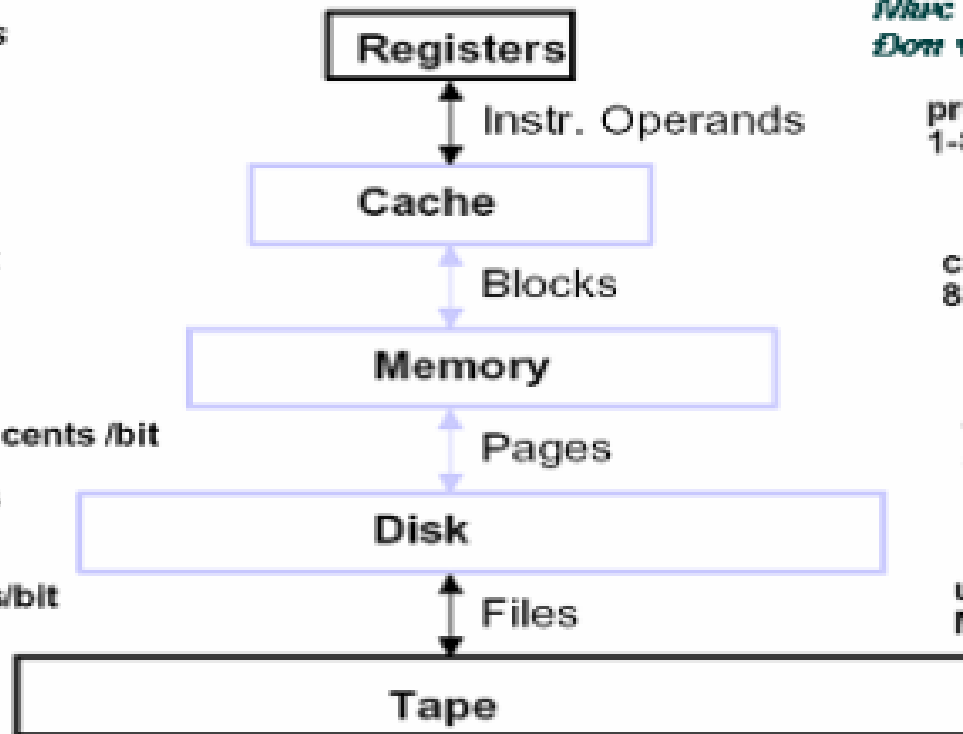
CPU Registers
100s Bytes
1s ns

Cache
K Bytes
4 ns
1-0.1 cents/bit

Main Memory
M Bytes
100ns- 300ns
\$.0001-.00001 cents /bit

Disk
G Bytes, 5 ms
(5,000,000 ns)
-5 -6
10 - 10 cents/bit

Tape
Infinite
sec./min
10



Mức quản lý
Đơn vị tham chiếu

prog./compiler
1-8 bytes

cache cntl
8-128 bytes

OS
512-4K bytes

user/operator
Mbytes

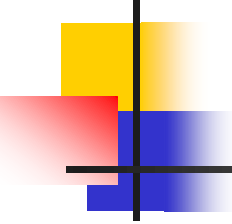
Mức cao

Nhanh hơn

Lớn hơn

Mức thấp

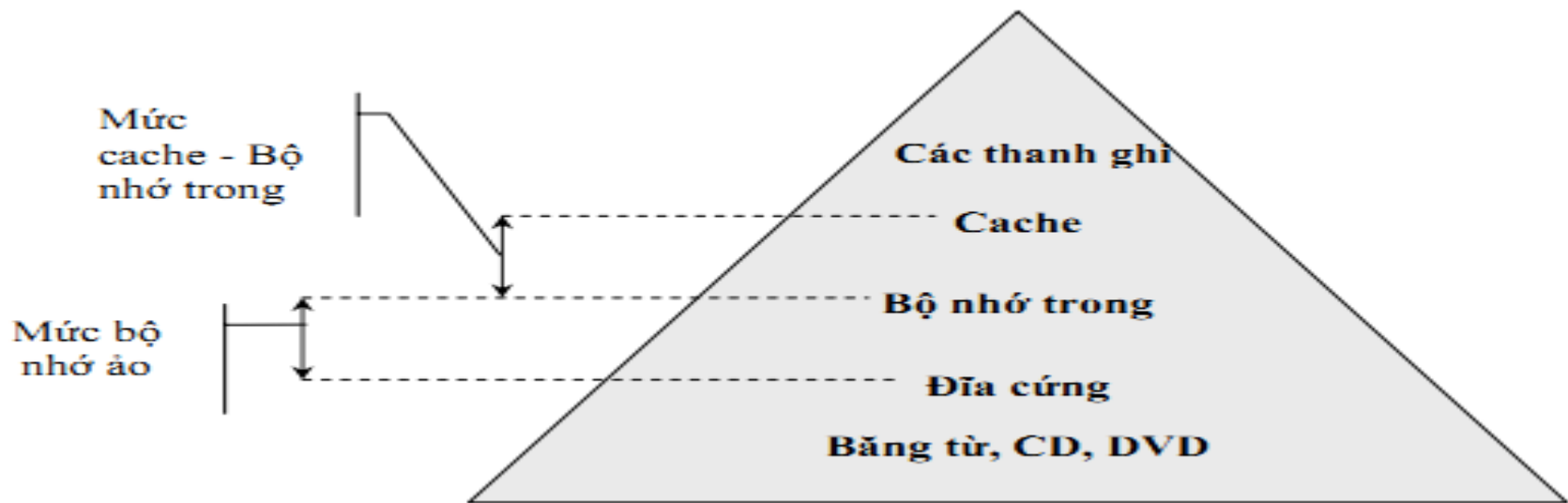
Các cấp bộ nhớ



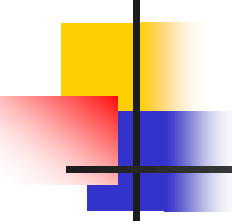
6.1. Các cấp bộ nhớ

Các đặc tính chính của các cấp bộ nhớ dẫn đến hai mức chính là: mức cache – bộ nhớ trong và mức bộ nhớ ảo (bao gồm bộ nhớ trong và không gian cấp phát trên đĩa cứng). Cách tổ chức này trong suốt đối với người sử dụng. Người sử dụng chỉ thấy duy nhất một không gian định vị ô nhớ, độc lập với vị trí thực tế của các lệnh và dữ liệu cần tham nhập.

6.1. Các cấp bộ nhớ

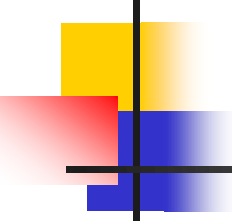


Hai mức bộ nhớ



6.1. Các cấp bộ nhớ

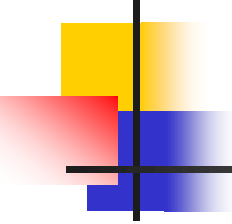
Nhận xét: “*mỗi cấp bộ nhớ có dung lượng lớn hơn cấp trên mình, ánh xạ một phần địa chỉ các ô nhớ của mình vào địa chỉ ô nhớ của cấp trên trực tiếp có tốc độ nhanh hơn, và các cấp bộ nhớ phải có cơ chế quản lý và kiểm tra các địa chỉ ánh xạ*”.



6.2. Khái niệm các loại bộ nhớ

6.2.1. Bộ nhớ cache

6.2.2. Bộ nhớ bán dẫn



6.2.1. Bộ nhớ cache

6.2.1.1. Khái niệm

6.2.1.2. Thuật ngữ khi sử dụng cache

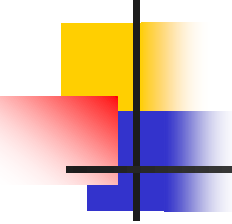
6.2.1.3. Đặc trưng cache

6.2.1.4. Các thành phần của Cache

6.2.1.5. Cấu trúc của cache

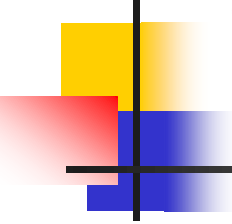
6.2.1.6. Các mức của cache

6.2.1.7. Hiệu quả của cache



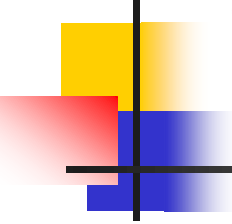
6.2.1.1. Khái niệm

Cache là một loại bộ nhớ có kích thước nhỏ và *tốc độ cao* thường là RAM tĩnh (SRAM – Static Random Access Memory) chứa các phần được truy cập gần nhất của bộ nhớ chính.



Vì sao Cache cần tốc độ cao?

Trong các hệ thống ngày nay, thời gian lấy để đưa ra một lệnh (hay một phần dữ liệu) trong CPU là rất dài khi so sánh với thời gian thực hiện lệnh. Nghĩa là, thời gian truy cập các bộ nhớ chậm hơn nhiều so với sự thực thi lệnh của CPU

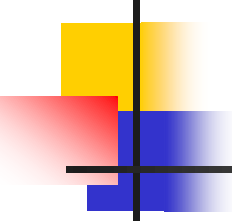


Vì sao Cache cần tốc độ cao?

. Ví dụ như, thời gian truy cập đặc trưng cho DRAM là 60ns, trong khi một CPU 100Mhz có thể thực hiện một lệnh trong một chu kỳ xung clock hay 10ns. Vì vậy sẽ xảy ra hiện tượng nghẽn cổ chai (bottle neck) ở ngõ vào CPU. Điều này làm CPU bị hạn chế về hiệu suất và tốc độ xử lý thực. Vì vậy cần có một bộ nhớ tốc độ cao để nâng cao hiệu suất chung cho hệ thống và đó chính là Cache. Cache thường là SRAM vì thời gian truy cập đặc trưng của SRAM nhanh, cỡ 15ns. Cache xem như là một phần nhỏ của bộ nhớ chính nhưng tốc độ truy cập thì nhanh hơn 3 hoặc 4 lần bộ nhớ chính (DRAM).

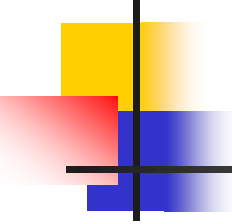
Vì sao một bộ nhớ nhỏ tốc độ cao có thể làm tăng hiệu suất hệ thống?

- Lý thuyết giải thích điều này gọi là “Vị trí tham chiếu” (Locality of Reference). Khái niệm này là tại bất kỳ một thời điểm nào được phép, CPU đang truy cập bộ nhớ trong một vùng nhỏ hoặc vùng định vị sẵn. Cache sẽ nạp vùng này và cho phép CPU truy xuất vùng nhớ nhanh hơn. Trong một ứng dụng điển hình, bộ nhớ Cache nội của vi xử lý Pentium là 16K-byte có thể chứa trên 90% các địa chỉ được yêu cầu bởi nhân vi xử lý. Nghĩa là, Hơn 90% sự truy cập bộ nhớ xuất hiện trong bộ nhớ Cache tốc độ cao.



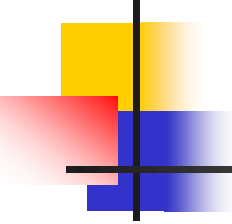
Vì sao một bộ nhớ nhỏ tốc độ cao có thể làm tăng hiệu suất hệ thống?

- Tuy nhiên, hiệu suất của bộ nhớ Cache liên quan trực tiếp đến các ứng dụng chạy trên hệ thống. Thông số này dựa trên việc chạy các ứng dụng trên máy tính đề bàn chuẩn. Hiệu suất Cache có thể thấp hơn phụ thuộc vào phần mềm được thiết kế.
- Cache nằm giữa CPU và bộ nhớ trong nhằm tăng tốc độ truy cập bộ nhớ của CPU.
- Cache có thể đặt trên chip của CPU.



6.2.1.2. Các thuật ngữ khi sử dụng cache

- **Thành công cache (cache hit):** bộ xử lý tìm gặp phần tử cần đọc (ghi) trong cache.
- **Thất bại cache (cache miss):** bộ xử lý không gặp phần tử cần đọc (ghi) trong cache
- **Trừng phạt thất bại cache (cache penalty):** Thời gian cần thiết để xử lý một thất bại cache. Thời gian bao gồm thời gian thâm nhập bộ nhớ trong cộng với thời gian chuyển khối chứa từ cần đọc từ bộ nhớ đến cache. Thời gian này tùy thuộc vào kích thước khối.

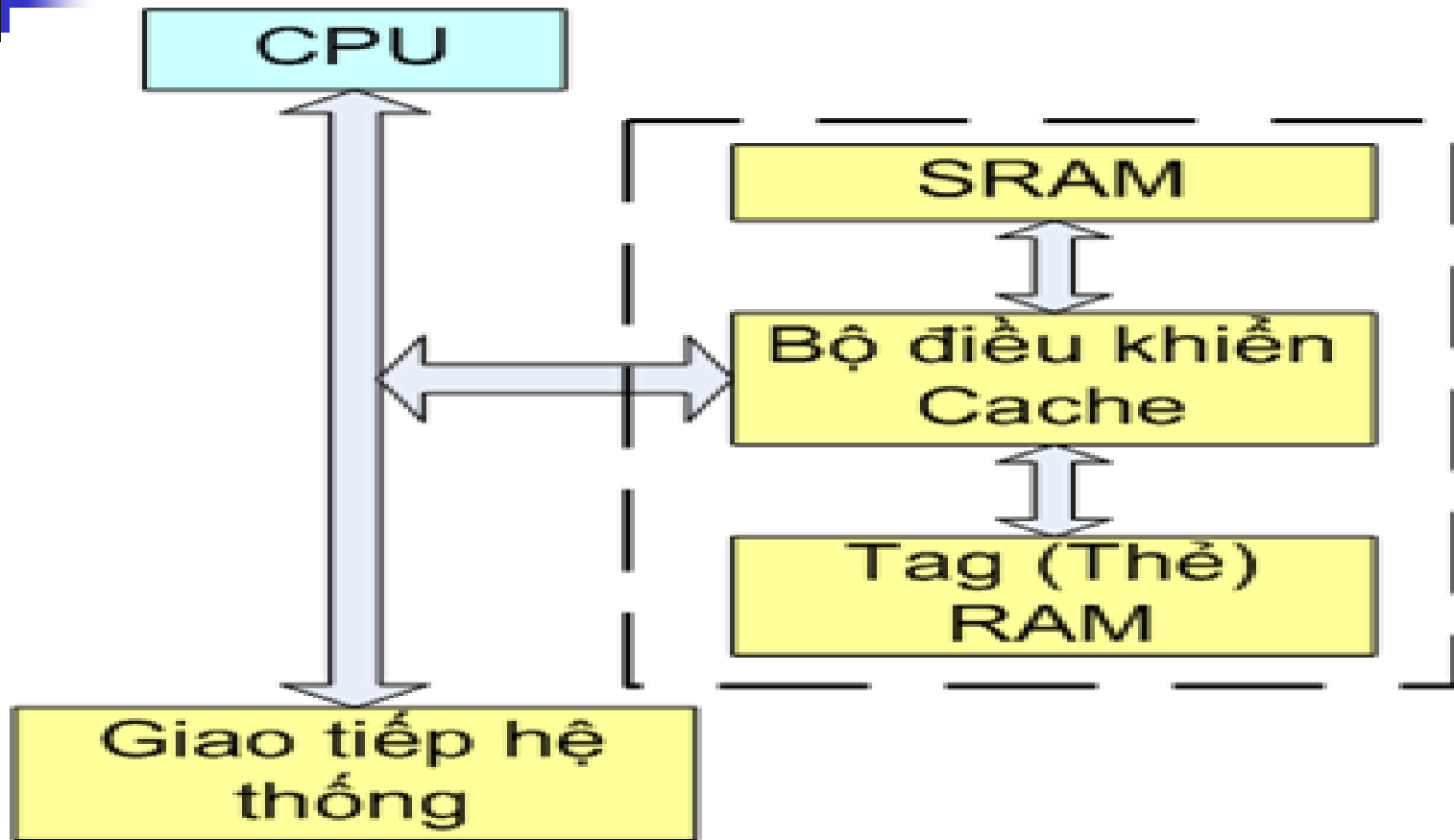


6.2.1.3. Các đặc trưng cache

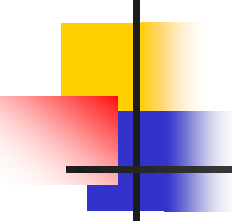
- Các Cache có hai đặc trưng qua trọng là cấu trúc đọc (Read architecture) và cách ghi (Write policy).
- Cấu trúc đọc có thể là “Look Aside” hoặc “Look Through”. Cấu trúc ghi có thể là “Write-Back” hoặc “Write-Through”. Cả hai loại cấu trúc đọc có thể sử dụng cùng một trong hai cách ghi tùy vào thiết kế.

6.2.1.3. Các đặc trưng cache

Độc look aside



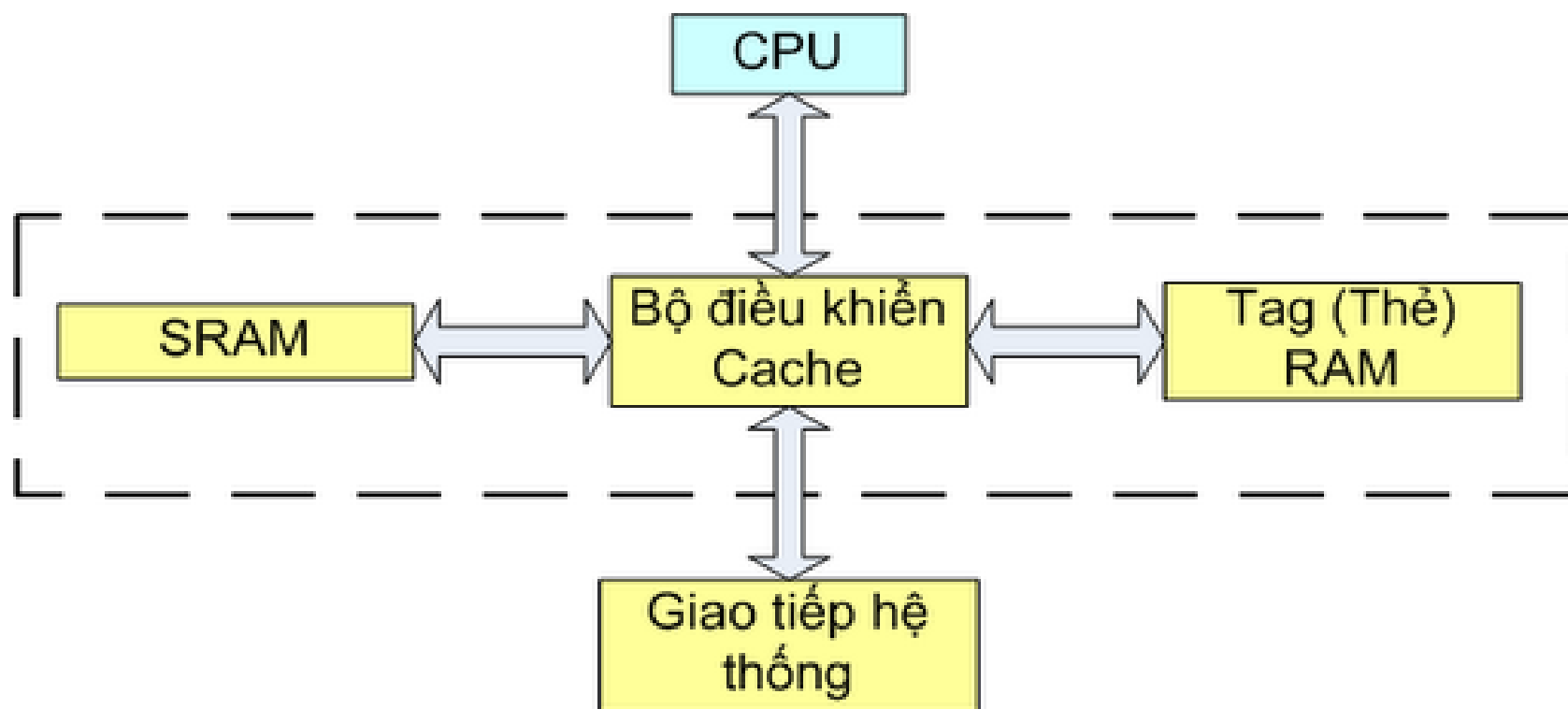
Cấu trúc độc look aside



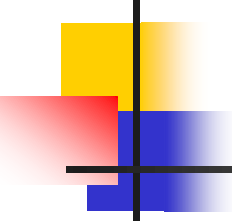
6.2.1.3. Các đặc trưng cache

Cache có cấu trúc Look Aside ít phức tạp nên giá thành thấp. Cấu trúc này cung cấp một đáp ứng tốt hơn đối với trường hợp “Cache miss” do cả DRAM và Cache (trong hình là SRAM) cùng “nhìn thấy” một chu kỳ bus trong cùng một thời điểm. Nhưng qua hình vẽ ta thấy rằng CPU không thể truy cập Cache nếu có một Bus master (Bus chủ - thành phần có quyền điều khiển bus hệ thống) khác đang truy cập bộ nhớ chính.

6.2.1.3. Các đặc trưng cache đọc look through

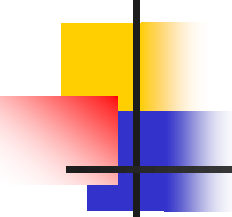


Cấu trúc đọc look through



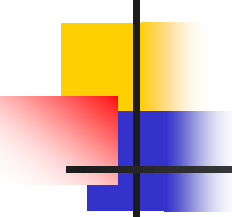
6.2.1.3. Các đặc trưng cache đọc look though

Bộ nhớ chính vẫn có vị trí đối diện với giao diện hệ thống. Ta dễ nhận ra là đơn vị Cache này là nó nằm giữa CPU và bộ nhớ chính. Điều quan trọng cần chú ý là Cache nằm giữa CPU và bộ nhớ chính và nó “thấy” chu kỳ bus CPU trước khi cho phép đi tiếp đến bus hệ thống.



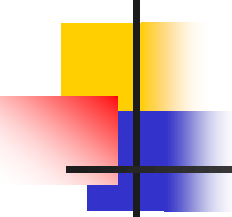
6.2.1.3. Các đặc trưng cache đọc look though

Cấu trúc này cho phép CPU chạy với Cache trong khi Bus master khác vẫn truy cập bộ nhớ chính, khi đó CPU tách riêng khỏi phần còn lại của hệ thống. Tuy nhiên, cấu trúc Cache này phức tạp hơn do phải kiểm soát các truy cập phần đến phần còn lại của hệ thống. Điều này làm tăng giá thành sản phẩm. Một điểm yếu khác là sự truy cập bộ nhớ khi “Cache miss” sẽ chậm hơn vì bộ nhớ chính sẽ không được truy cập cho đến khi Cache kiểm tra xong. Điều này sẽ không là vấn đề nếu Cache có tỷ lệ (hay tốc độ) HIT cao và các truy cập bộ nhớ chính là của các Bus master khác.



6.2.1.3. Các đặc trưng cache cách ghi

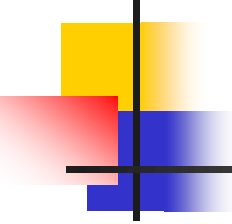
Write-Back: Cache hoạt động như một bộ đệm. Tức là, khi CPU bắt đầu một chu trình ghi, Cache sẽ nhận dữ liệu và kết thúc chu trình này. Cache sẽ ghi lại dữ liệu vào bộ nhớ chính khi bus hệ thống sẵn sàng. Phương pháp này cung cấp hiệu suất lớn nhất bằng cách cho phép CPU tiếp tục xử lý các nhiệm vụ trong khi bộ nhớ chính được cập nhật tại thời điểm sau đó. Tuy nhiên, việc điều khiển ghi như vậy sẽ làm tăng sự phức tạp của Cache và giá thành thực hiện.



6.2.1.3. Các đặc trưng cache

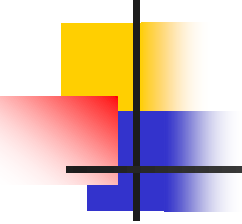
Cách ghi

Write-Through: Giống như tên gọi, CPU ghi thông qua Cache đến bộ nhớ chính. Cache có thể cập nhật các nội dung của nó, tuy nhiên chu kỳ ghi sẽ không kết thúc cho đến khi dữ liệu được lưu trong bộ nhớ chính. Phương pháp này ít phức tạp hơn nên sẽ giảm giá thành khi thực hiện. Nhưng hiệu suất sẽ thấp hơn vì CPU phải chờ bộ nhớ Cache chấp nhận dữ liệu.

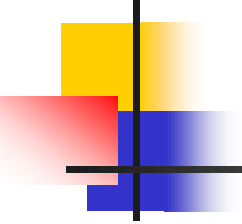


6.2.1.4. Các thành phần của cache

- Hệ thống Cache có thể được chia làm 3 khối chức năng là SRAM, tag RAM và Cache controller. Trong các thiết kế thực tế, các khối này có thể được thực hiện bởi nhiều chip hoặc được kết hợp chung trong một chip.
- **SRAM: Bộ nhớ truy cập ngẫu nhiên (Static Random Access Memory):** là khối lưu trữ dữ liệu, kích thước của SRAM chỉ ra kích thước của cache.
- **Tag RAM:** là một phần nhỏ của SRAM để lưu trữ địa chỉ của dữ liệu được lưu trong SRAM.

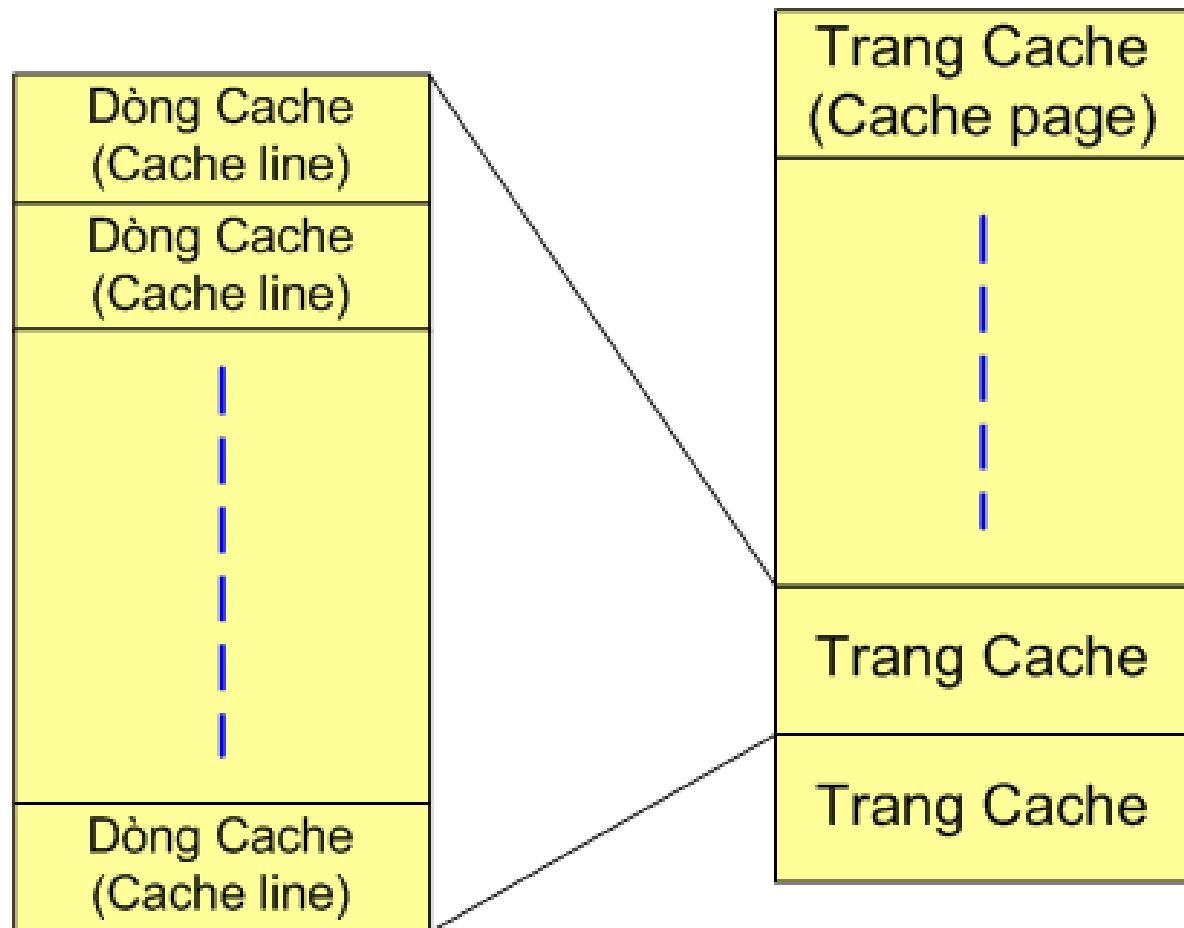


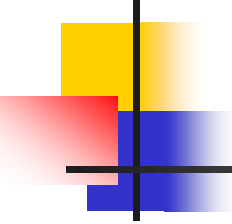
Bộ điều khiển cache (Cache controller): Bộ điều khiển cache được xem như bộ não của cache. Các nhiệm vụ nó đảm nhận là thực thi các snoop và snarf, cập nhật SRAM và Tag RAM. Thực thi quy định ghi, bộ điều khiển cache còn có nhiệm vụ xác định yêu cầu bộ nhớ có thể “cache” nghĩa là vùng bộ nhớ nào được lưu trong cache và các yêu cầu ‘cache hit’ và ‘cache miss’.



Lưu ý: Không phải vùng nào cũng được lưu trong cache. Vùng được lưu gọi là Cacheable, Vùng nào bộ nhớ chính xác định là không thể lưu trong cache gọi là non-cacheable, điều này tùy thuộc vào thiết kế. Ví dụ trong máy tính thì Video không được lưu trong cache.

6.2.1.5. Cấu trúc của cache



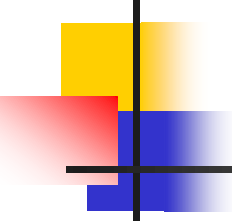


Để hiểu đầy đủ cấu tạo của Cache, ta cần nắm hai thuật ngữ Cache page và Cache line.

- *Cache page* : Một bộ nhớ chính được chia làm nhiều phần bằng nhau gọi là Cache page. Chú ý rằng, một Cache page không liên quan gì đến một trang bộ nhớ (Memory page) trong chế độ page. Trong cấu trúc PCU, Word page có nhiều ý nghĩa khác. Kích thước của page phụ thuộc kích thước và sự sắp xếp của Cache.

- *Cache line* : Một Cache page được cắt thành những phần nhỏ gọi là Cache line. Kích thước cache line được xác định bởi cả CPU và thiết kế của Cache.

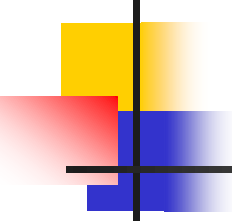
Hình 1 sẽ minh họa việc chia bộ nhớ chính thành các Cache page và các Cache page được chia thành các Cache line.



6.2.1.6 Các mức của cache

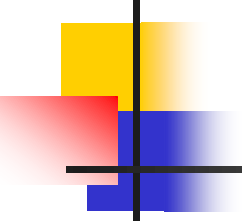
- Việc dùng cache trong có thể làm cho sự cách biệt giữa kích thước và thời gian thâm nhập giữa cache trong và bộ nhớ trong ngày càng lớn. Người ta đưa vào nhiều mức cache:
- Cache mức một (L1): thường là cache trong (on-chip cache) nằm bên trong CPU
- Cache mức hai (L2): thường là cache ngoài (off-chip cache) nằm bên ngoài CPU.
- Ngoài ra một số hệ thống còn có cache mức ba (L3): cache trung gian giữa cache L2 và bộ nhớ [!:-27-:]

Bộ xử lý	Kiểu	Năm phát hành	L1 Cache ^a	L2 Cache	L3 Cache
IBM 360/85	Mainframe	1968	16 to 32 KB	-	-
PDP-11/70	Mini Computer	1975	1 KB	-	-
VAX 11/780	Mini Computer	1978	16 KB	-	-
IBM 3033	Mainframe	1978	64 KB	-	-
IBM 3090	Mainframe	1985	128 to 256 KB	-	-
Intel 80486	PC	1989	8 KB	-	-
Pentium	PC	1993	8 KB / 8 KB	256 to 512 KB	-
PowerPC 601	PC	1993	32 KB	-	-
PowerPC 620	PC	1996	32 KB / 32 KB	-	-
PowerPC G4	PC/Server	1999	32 KB / 32 KB	256KB to 1MB	2 MB
IBM S390/G4	Mainframe	1997	32 KB	256 KB	2 MB
IBM S390/G6	Mainframe	1999	256 KB	8 MB	-
Pentium 4	PC/Server	2000	8 KB / 8 KB	256 KB	-
IBM SP	High-End server/ Super Computer	2000	64 KB / 32 KB	8 MB	-
CRAY MTA ^b	Super Computer	2000	8 KB	2 MB	-
Itanium	PC/Server	2001	16 KB / 16 KB	96 KB	2 MB
SGI Origin 2001	High-End server	2001	32 KB / 32 KB	4 MB	-

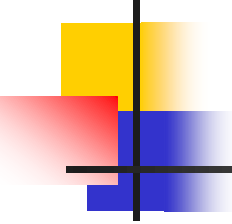


6.2.1.7. Hiệu quả cache

- Thông thường người ta dùng thời gian thâm nhập trung bình bộ nhớ trong để đánh giá hiệu quả của cache.
- Thời gian thâm nhập trung bình được cho bởi công thức:
- $(\text{thời gian thâm nhập trung bình bộ nhớ}) = (\text{thời gian thâm nhập thành công}) + (\text{tỉ số thất bại}) * (\text{trùng phạt thất bại})$.

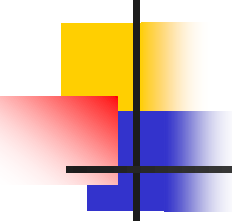


Thời gian thâm nhập thành công là thời gian thâm nhập vào một thông tin trong một thành công cache. Tỷ số thất bại là tỷ số giữa số thất bại cache và tổng số thâm nhập cache. Thời gian thâm nhập thành công và trừng phạt thất bại được đo bằng đơn vị thời gian hoặc bằng chu kỳ xung nhịp (clock cycle).

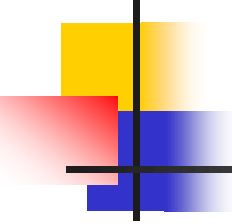


Thất bại cache???

- **Khởi động:** trong lần thâm nhập cache đầu tiên, không có thông tin cần tìm trong cache nên phải chuyển khối chứa thông tin đó vào cache.
- **Khả năng:** vì cache không thể chứa tất cả các khối cần thiết cho việc thi hành một chương trình nên gặp thất bại do cache thiếu khả năng, do đó một khối bị lấy ra khỏi cache rồi lại được đưa vào sau này.
- **Tranh chấp:** Nếu chiến thuật thay thế các khối là phối hợp theo tập hợp hay tương ứng trực tiếp, các thất bại do tranh chấp xảy ra vì một khối có thể bị đưa ra khỏi cache rồi được gọi vào sau đó nếu có nhiều khối phải được thay thế trong các tập hợp.

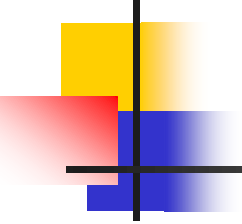


6.2.2. Bộ nhớ bán dẫn



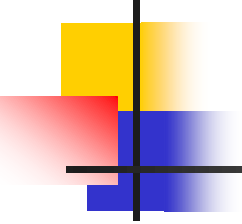
6.2.2.1. Bộ nhớ ROM (Read Only Memory)

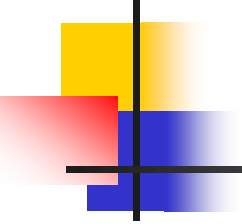
- *ROM là bộ nhớ chỉ đọc, là bộ nhớ không khả biến.*
- *Bộ nhớ ROM cũng được chế tạo bằng công nghệ bán dẫn. Chương trình trong ROM được viết vào lúc chế tạo nó. Thông thường, ROM chứa chương trình khởi động máy tính, chương trình điều khiển trong các thiết bị điều khiển tự động...*
- *ROM sử dụng để lưu các thông tin sau:*
- *Thư viện các chương trình con.*
- *Các chương trình con điều khiển hệ thống (BIOS)*
- *Các bảng chức năng.*

- 
- **PROM (Programable ROM):** chế tạo bằng các mối nối (cầu chì – có thể làm đứt bằng điện). Chương trình nằm trong PROM có thể được viết vào bởi người sử dụng bằng thiết bị đặc biệt và không thể xóa bỏ.
 - **EPROM (Erasable Programable ROM):** Chế tạo bằng nguyên tắc phân cực tĩnh điện. Chương trình nằm trong ROM có thể được viết vào (bằng điện) và có thể xóa (bằng tia cực tím – trung hòa tĩnh điện) để viết lại bởi người sử dụng.
 - **EEPROM (Electrically Erasable Programable ROM):** Chế tạo bằng công nghệ bán dẫn. Chương trình nằm trong ROM có thể được viết vào và có thể xóa (bằng điện) để viết lại bởi người dùng.

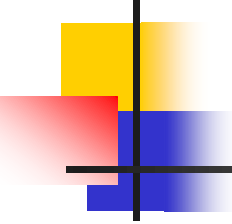
6.2.2.2. Bộ nhớ RAM (Random Access Memory)

- RAM là bộ nhớ truy cập ngẫu nhiên.
- Bộ nhớ RAM có đặc tính là các ô nhớ có thể được đọc hoặc viết vào trong khoảng thời gian bằng nhau cho dù chúng ở bất kỳ vị trí nào trong bộ nhớ. Mỗi ô nhớ có một địa chỉ, thông thường, mỗi ô nhớ là một byte (8bit), nhưng hệ thống có thể đọc ra hay viết vào nhiều byte (2, 4, 8byte). Bộ nhớ trong RAM được đặc trưng bằng dung lượng và tổ chức của nó (số ô nhớ và số bit cho mỗi ô nhớ), thời gian thâm nhập (thời gian từ lúc đưa ra địa chỉ ô nhớ đến lúc đọc được nội dung ô nhớ đó) và chu kỳ bộ nhớ (thời gian giữa hai lần liên tiếp thâm nhập bộ nhớ).

- 
- Tùy theo công nghệ chế tạo, người ta phân biệt RAM tĩnh (SRAM: Static RAM) và RAM động (Dynamic RAM).
 - **RAM tĩnh:** Được chế tạo theo công nghệ ECL (CMOS và BiCMOS). Mỗi bit nhớ gồm có các cổng logic với độ 6 transistor MOS, việc nhớ một dữ liệu là tồn tại nếu bộ nhớ được cung cấp điện. SRAM là bộ nhớ nhanh, việc đọc không làm hủy nội dung của ô nhớ và thời gian thâm nhập bằng chu kỳ bộ nhớ.



RAM động: Dùng kỹ thuật MOS. Mỗi bit nhớ gồm có một transistor và một tụ điện. Cũng như SRAM, việc nhớ một dữ liệu là tồn tại nếu bộ nhớ được cung cấp điện. Việc ghi nhớ đưa vào việc duy trì điện tích nạp vào tụ điện và như vậy đọc một bit nhớ làm nội dung bit này bị hủy. Vậy sau mỗi lần đọc một bộ nhớ, bộ phận điều khiển bộ nhớ phải viết lại ô nhớ đó nội dung vừa đọc và do đó chu kỳ bộ nhớ động ít nhất là gấp đôi thời gian thâm nhập bộ nhớ.



Các RAM thường gặp

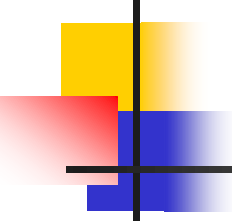
SRAM

DRAM

SDRAM

DDRAM

RDRAM



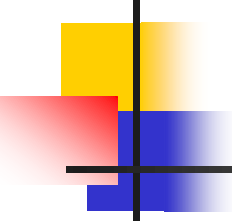
6.3. Mức cache – bộ nhớ trong

Phải để một khối bộ nhớ vào chỗ nào của cache (sắp xếp khối)?

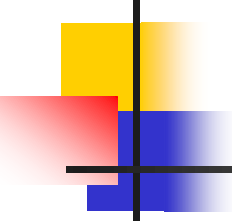
Làm sao tìm một khối khi nó hiện diện trong cache (nhận diện khối)?

Khối nào phải được thay thế trong trường hợp thất bại cache (thay thế khối)?

Việc gì xảy ra khi ghi vào bộ nhớ (chiến thuật ghi)?



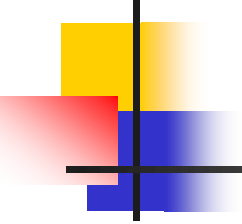
Mức cache – bộ nhớ trong trong bảng các cấp bộ nhớ có cơ cấu vận hành trong suốt đối với bộ xử lý. Với thao tác đọc bộ nhớ, bộ xử lý gửi một địa chỉ và nhận một dữ liệu từ bộ nhớ trong. Với thao tác ghi bộ nhớ viết một dữ liệu vào một ô nhớ với một địa chỉ được chỉ ra trong bộ nhớ. Để cho chương trình vận hành bình thường thì cache phải chứa một phần con của bộ nhớ trong để bộ xử lý có thể thâm nhập vào các lệnh hoặc dữ liệu thường dùng từ bộ nhớ cache.

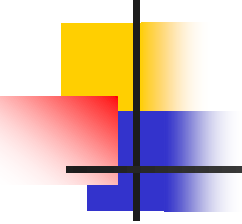


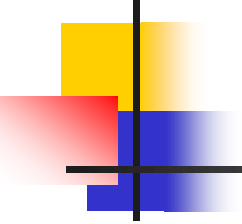
Do dung lượng của bộ nhớ cache nhỏ nên nó chỉ chứa một phần chương trình nằm trong bộ nhớ trong. Để đảm bảo sự đồng nhất giữa nội dung của cache và bộ nhớ trong thì cache và bộ nhớ trong phải có cùng cấu trúc. Việc chuyển dữ liệu giữa cache và bộ nhớ trong là việc tải lên hay ghi xuống các khối dữ liệu. Mỗi khối chứa nhiều từ bộ nhớ tùy thuộc vào cấu trúc bộ nhớ cache. Sự lựa chọn kích thước của khối rất quan trọng cho vận hành của cache có hiệu quả.

Phải để một khối bộ nhớ vào chỗ nào của cache (sắp xếp khối)?

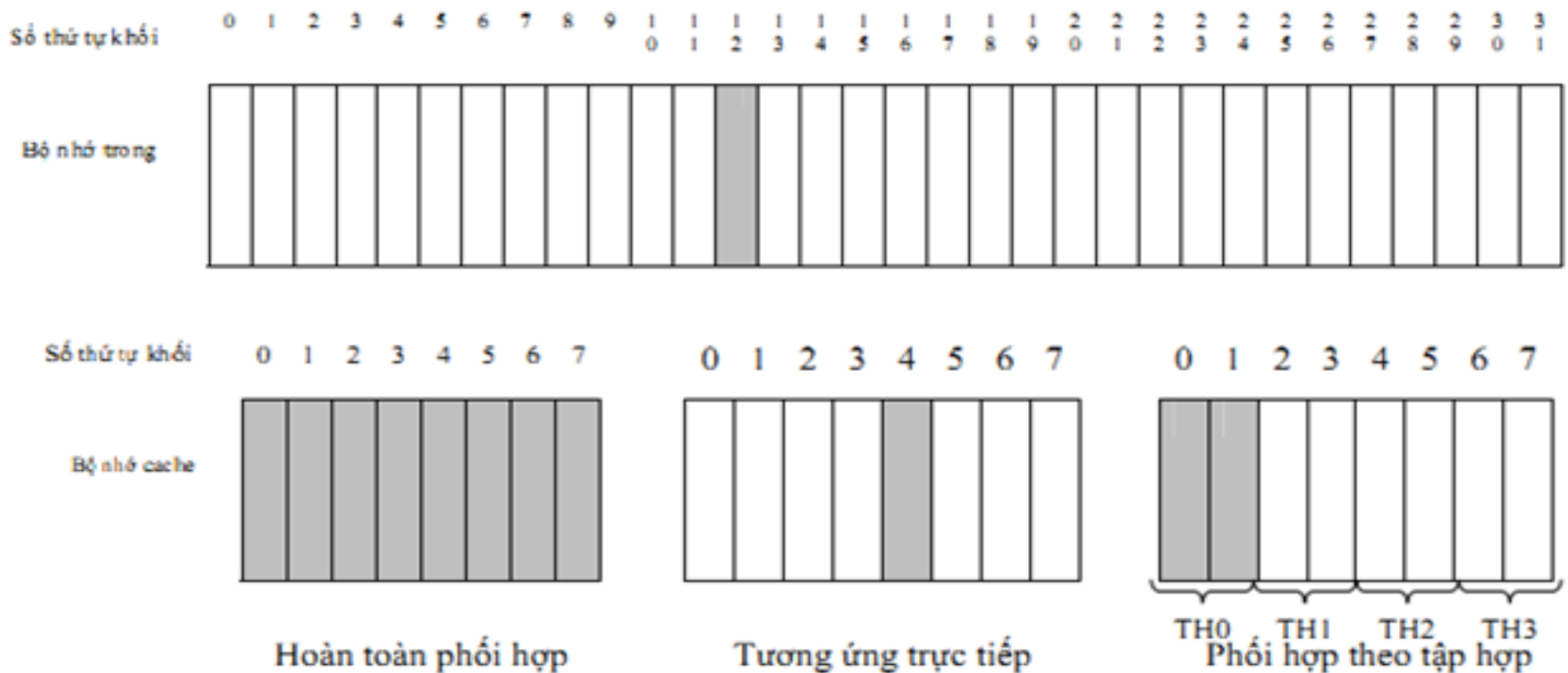
- *Kiểu tương ứng trực tiếp*: Nếu mỗi khối bộ nhớ chỉ có một vị trí đặt khối duy nhất trong cache được xác định theo công thức $K = i \bmod n$. Trong đó:
 - K: vị trí khối đặt trong cache
 - I: số thứ tự của khối trong bộ nhớ trong
 - N: số khối của cache
- Như vậy, trong kiểu xếp đặt khối này, mỗi vị trí đặt khối trong cache có thể chứa một trong các khối trong bộ nhớ cách nhau xn khối (x: 0,1,...,m; n: số khối của cache).

- 
-
- *Kiểu hoàn toàn phối hợp*: trong kiểu đặt khối này, một khối trong bộ nhớ trong có thể được đặt vào vị trí bất kỳ trong cache.
 - Như vậy, trong kiểu xếp đặt khối này, mỗi vị trí đặt khối trong cache có thể chứa một trong tất cả các khối trong bộ nhớ.

- 
- *Kiểu phối hợp theo tập hợp:* với cách tổ chức này, cache bao gồm các tập hợp của các khối cache. Mỗi tập hợp của các khối cache chứa số khối như nhau. Một khối của bộ nhớ trong có thể được đặt vào một số vị trí khối giới hạn trong tập hợp được xác định bởi công thức: $K = i \bmod s$
 - Trong đó:
 - K: vị trí khối đặt trong cache
 - i: số thứ tự của khối trong bộ nhớ trong
 - s: số lượng tập hợp trong cache
 -

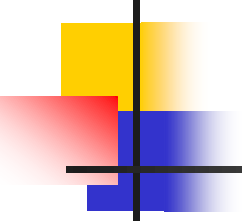
- 
- Trong cách đặt khối theo kiểu phối hợp theo tập hợp, nếu tập hợp có m khối, sự tương ứng giữa các khối trong bộ nhớ trong và các khối của cache được gọi là phối hợp theo tập hợp m khối.
 - Nếu $m = 1$ (mỗi tập hợp có 1 khối), ta có kiểu tương ứng trực tiếp.
 - Nếu $m = n$ (n : số khối của cache), ta có kiểu tương hoàn toàn phối hợp.
 - Hiện nay, phần lớn các cache của các bộ xử lý đều là kiểu tương ứng trực tiếp hay kiểu phối hợp theo tập hợp (mỗi tập hợp gồm 2 hoặc 4 khối).

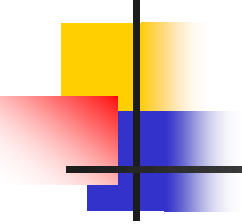
Ví dụ: Bộ nhớ trong có 32 khối, cache có 8 khối, mỗi khối gồm 32byte, khối thứ 12 của bộ nhớ trong được đưa vào cache.



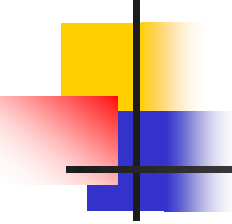
Khối nào phải được thay thế trong trường hợp thất bại cache (thay thế khối)?

Khi có thất bại cache, bộ điều khiển cache thêm nhập bộ nhớ trong và chuyển khối mà bộ xử lý cần đọc (ghi) vào cache. Như vậy, khối nào trong cache sẽ bị thay thế bởi khối mới được chuyển lên. Đối với kiểu tương ứng trực tiếp, vị trí đặt khối không có sự lựa chọn, nó được xác định bởi trường chỉ số khối cache trong địa chỉ của từ cần đọc (ghi). Nếu cache là kiểu hoàn toàn phối hợp hay phối hợp theo tập hợp thì khi thất bại phải chọn lựa thay thế theo khối. **Có bốn chiến thuật chủ yếu dùng để chọn khối thay thế trong cache:**

- 
- **Thay thế ngẫu nhiên**: để phân bố đồng đều việc thay thế, các khối cần thay thế trong cache được chọn ngẫu nhiên.
 - **Khối xưa nhất** (LRU: Least Recently Used): các khối đã được thêm nhập sẽ được đánh dấu và khối bị thay thế là khối không được dùng từ lâu nhất.
 - **Vào trước ra trước** (FIFO): Khối được đưa vào cache đầu tiên, nếu bị thay thế, khối đó sẽ được thay thế trước nhất.
 - **Tần số sử dụng ít nhất**: (LFU: least Frequently Used): Khối trong cache được tham chiếu ít nhất.



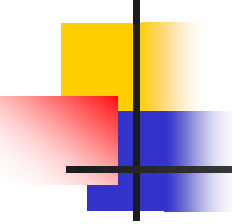
Điều này sử dụng hệ quả của nguyên tắc sử dụng ô nhớ theo thời gian: nếu các khối mới được dùng có khả năng sẽ được dùng trong tương lai gần, khối bị thay thế là khối không dùng trong thời gian lâu nhất.



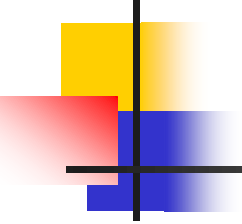
Việc gì xảy ra khi ghi vào bộ nhớ (Chiến thuật ghi)???

Có 2 cách ghi đồng nhất dữ liệu lưu trữ:

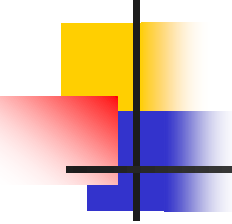
- Ghi đồng thời
- Ghi lại



Ghi đồng thời: Thông tin được ghi đồng thời vào khối của cache và khối của bộ nhớ trong. Cách ghi này làm chậm tốc độ chung của hệ thống. Các ngoại vi có thể truy cập bộ nhớ trực tiếp.



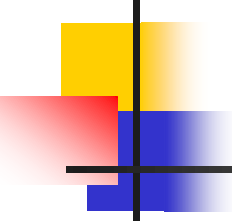
Ghi lại: Để đảm bảo tốc độ xử lý của hệ thống, thông tin cần ghi chỉ được ghi vào khối trong cache. Để quản lý sự khác biệt nội dung giữa khối của cache và khối của bộ nhớ trong, một bit trạng thái (Dirty bit hay Update bit) được dùng để chỉ thị. Khi một thao tác ghi vào trong cache, bit trạng thái của khối cache sẽ được thiết lập. Khi một khối bị thay thế, khối này sẽ được ghi lại vào bộ nhớ trong chỉ khi bit trạng thái đã được thiết lập. Với cách ghi này, các ngoại vi liên hệ đến bộ nhớ trong thông qua cache.



Khi có một thất bại ghi vào cache thì phải lựa chọn một trong hai giải pháp sau:

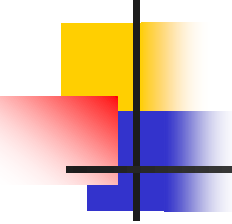
Ghi có nạp: Khối cần ghi từ bộ nhớ trong được nạp vào trong cache như mô tả ở trên. Cách này thường được dùng trong cách ghi lại.

Ghi không nạp: Khối được thay đổi ở bộ nhớ trong không được đưa vào cache. Cách này dùng trong cách ghi đồng thời.



6.4. Bộ nhớ ảo

- Bộ nhớ ảo xác định một cơ chế vận chuyển tự động số liệu giữa bộ nhớ trong và bộ nhớ ngoài (đĩa từ).
- Trước đây, khi độ dài của chương trình vượt quá giới hạn dung lượng bộ nhớ thì người lập trình phải phân chia chương trình của mình thành từng phần tự loại bỏ nhau (overlays) và phải tự quản lý việc trao đổi thông tin giữa bộ nhớ và đĩa từ.
- Bộ nhớ ảo làm nhẹ trách nhiệm của các nhà lập trình bằng cách làm cho việc trao đổi thông tin này được thực hiện một cách tự động.



Trong các BXL hiện đại, bộ nhớ ảo được dùng để cho phép thực hiện cùng lúc nhiều tiến trình (process), mỗi tiến trình có một không gian định vị riêng. Nếu tất cả các không gian định vị này đều thuộc không gian định vị bộ nhớ trong thì rất tốn kém. Bộ nhớ ảo bao gồm bộ nhớ trong và bộ nhớ ngoài được phân tích thành khối để có thể cung cấp cho mỗi chương trình một số khối cần thiết cho việc thực hiện chương trình đó.

địa chỉ ảo

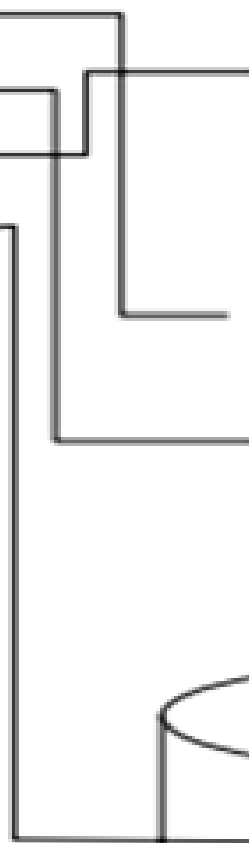
0	A
4K	B
8K	C
12K	D
16K	
20K	
24K	
28K	
32K	
...	...
...	...
16M	

địa chỉ vật lý

0	
4K	C
8K	
12K	
16K	A
20K	
24K	B
28K	

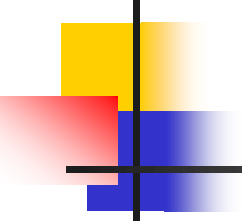
bộ nhớ trong

bộ nhớ ảo



Đĩa cứng

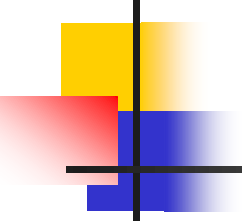
Một chương trình gồm 4 trang A,B,C,D trong đó D nằm trong ổ đĩa.



Ngoài việc phân chia không gian bộ nhớ, cần bảo vệ và quản lý tự động các cấp bộ nhớ, bộ nhớ ảo đơn giản hóa việc nạp chương trình vào bộ nhớ để thi hành một cơ chế gọi là *sự tái định địa chỉ* (address relocation). Cơ chế này cho phép một chương trình có thể được thi hành khi nó nằm ở bất cứ vị trí nào trong bộ nhớ.

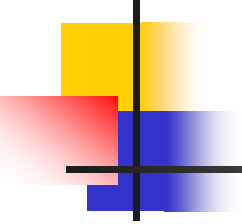
Tham số	Cache	Bộ nhớ ảo
Chiều dài mỗi khối (trang)	16-128byte	4096-65536byte
Thời gian thâm nhập thành công	1-2 xung nhịp	40-100 xung nhịp
Trùng phạt khi thất bại	8-100 xung nhịp	700.000 – 6 triệu xung
(Thời gian thâm nhập)	6-60 xung	500.000 – 4 triệu xung
(Di chuyển số liệu)	2-40 xung	200.000 – 2 triệu xung
Tỉ số thất bại	0,5%-10%	0,00001%-0,001%
Dung lượng	8KB – 8MB	16MB – 8GB

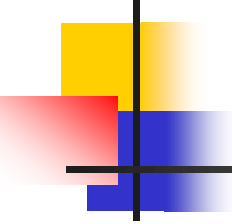
Đại lượng điển hình cho bộ nhớ cache và bộ nhớ ảo



Ngoài sự khác biệt định lượng mà ta thấy trong hình trên còn có những khác biệt giữa bộ nhớ cache và bộ nhớ ảo là:

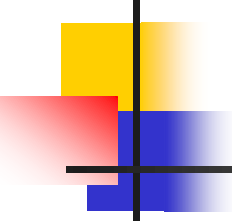
- Khi thất bại cache, sự thay thế một khối trong cache được điều khiển bằng phần cứng, còn trong bộ nhớ ảo chủ yếu do hệ điều hành.
- Không gian định vị mà BXL quản lý là không gian định vị của bộ nhớ ảo, trong lúc đó thì dung lượng bộ nhớ cache không tùy thuộc vào không gian định vị BXL.
- Bộ nhớ ngoài còn được dùng để lưu trữ tập tin ngoài nhiệm vụ là hậu phương của bộ nhớ trong (trong các cấp bộ nhớ).

- 
-
- Một khối được đặt ở đâu trong bộ nhớ trong?
 - Làm thế nào để tìm một khối khi nó đang nằm trong bộ nhớ trong?
 - Khối nào được thay thế khi có thất bại trang?
 - Việc gì xảy ra khi cần ghi số liệu?
 - Sự bảo vệ tiến trình bằng cách dùng bộ nhớ ảo.



Khối nào được thay thế khi có thất bại trang?

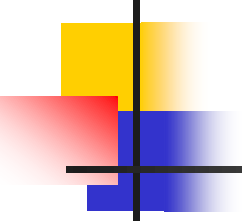
Hầu hết các hệ điều hành đều cố gắng thay thế khối ít dùng gần nhất (LRU: Least Recent Utilized) vì nghĩ rằng đây là khối ít cần nhất.

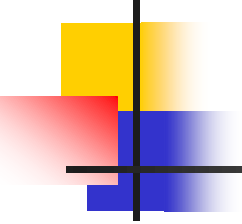


Việc gì xảy ra khi cần ghi số liệu?

Chiến thuật ghi luôn là một sự ghi lại nghĩa là thông tin chỉ được viết vào trong khối của bộ nhớ trong. Khối có thay đổi thông tin, được chép vào bộ nhớ ngoài. nếu khối này bị thay thế.

Sự bảo vệ tiến trình bằng cách dùng bộ nhớ ảo???

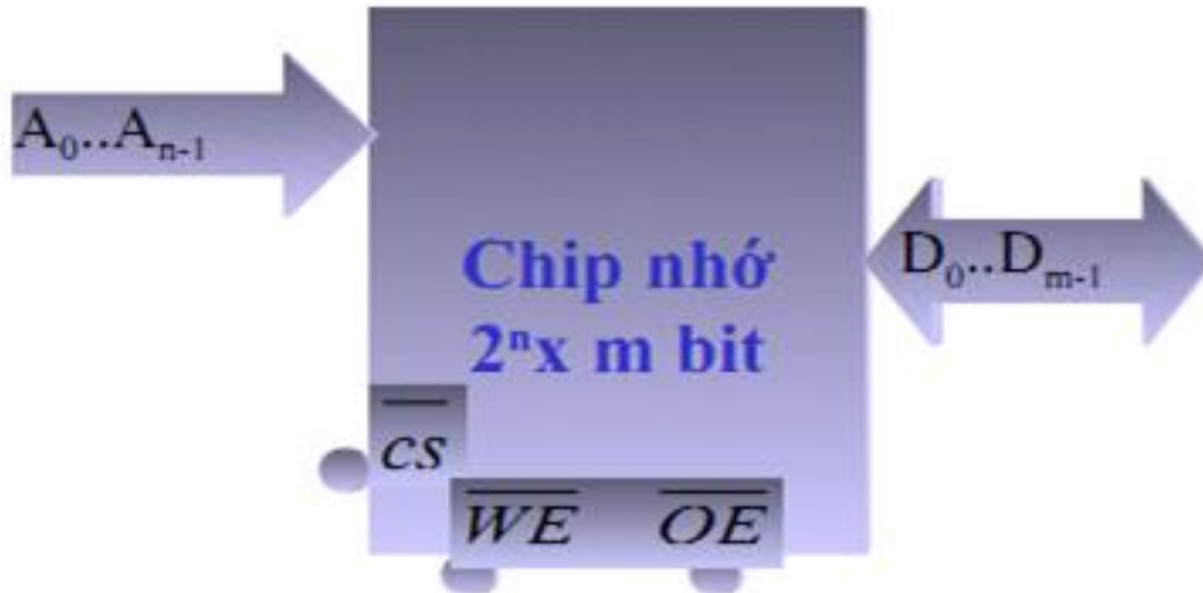
- 
- Sự xuất hiện của đa chương trình (multiprogram) trong đó máy tính chạy nhiều chương trình song song với nhau, dẫn đến các đòi hỏi mới về việc bảo vệ và phân chia giữa các chương trình.
 - Đa chương trình đưa đến khái niệm tiến trình (process): *một tiến trình gồm có một chương trình đang được thực hiện và tất cả các thông tin cần thiết để tiếp tục thực hiện chương trình này.*



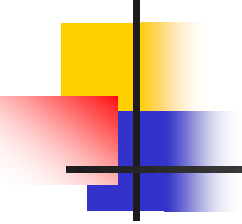
Nhà thiết kế máy tính có thêm 3 trách nhiệm trong việc giúp các nhà thiết kế hệ điều hành bảo vệ các tiến trình là:

- Cung cấp hai chế độ vận hành cho biết tiến trình đang thực hiện là tiến trình của người sử dụng hay tiến trình của hệ thống (của người điều hành).
- Cung cấp một tập hợp con trạng thái của BXL mà tiến trình người sử dụng có thể dùng nhưng không thể sửa đổi.
- Cung cấp các chế độ có thể chuyển đổi từ chế độ người sang chế độ điều hành và ngược lại.

6.5. Tổ chức chip nhớ



Sơ đồ tổ chức chip nhớ

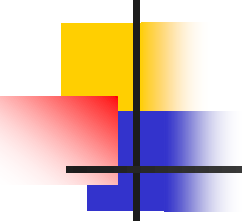
- 
-
- Các tín hiệu của chip nhớ
 - Các đường địa chỉ: $A_0 \dots A_{n-1}$ để xác định 2^n ngăn nhớ.
 - Các đường dữ liệu: $D_0 \dots D_{m-1}$ độ dài từ nhớ (m bit).
 - Dung lượng chip nhớ: $2^n \times m$ bit
 - Các tín hiệu điều khiển
 - Tín hiệu chọn chip hoạt động: CS (chip select)
 - Tín hiệu điều khiển đọc hoặc ghi (WE: Write Enable; OE: Output Enable).
 - Thường các tín hiệu điều khiển tích cực với mức 0.

Thiết kế module nhớ bán dẫn.

- Cho chip $2^n \times m$ bit
- Yêu cầu sử dụng chip nhớ trên thiết kế module nhớ dung lượng là bội kích thước chip nhớ trên.
- Giải quyết vấn đề
 - Thiết kế để tăng độ dài từ nhớ, số ngăn nhớ không thay đổi
 - Thiết kế để tăng số lượng ngăn nhớ, độ dài từ nhớ không thay đổi

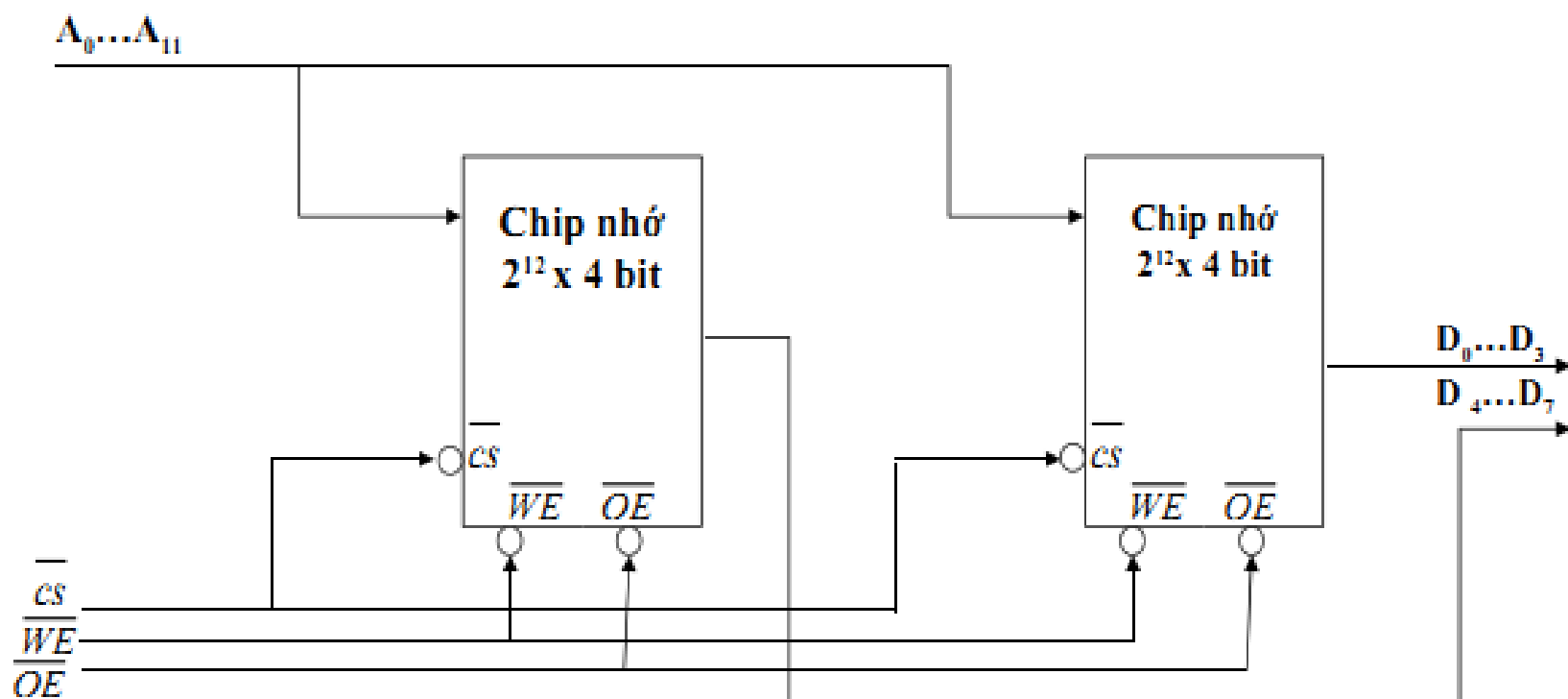
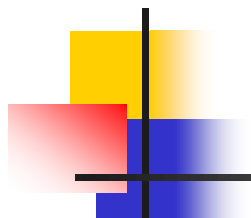
Thiết kế tăng số lượng từ nhớ

- Giả thiết: Cho các chip nhớ có dung lượng $2^n \times m$ bit.
- Yêu cầu: thiết kế module nhớ có kích thước $2^n \times (k.m)$ bit.
- Giải quyết:
- Để thiết kế được yêu cầu ta xác định hai thông số n (số đường địa chỉ) và k (số chip nhớ cần để ghép vào module thiết kế)



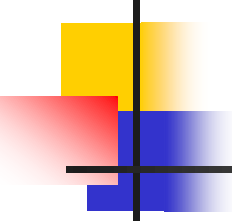
Ví dụ: Cho các chip nhớ SDRAM dung lượng 4K x 4bit. Hay thiết kế module nhớ có kích thước 4K x 8bit.

- Ta có: Dung lượng chip nhớ $2^{12} \times 4\text{bit} \Rightarrow$ Số đường địa chỉ $n = 12$, số đường dữ liệu $m = 4$.
- Thông tin về module nhớ số lượng địa chỉ là 12 đường (số ngăn nhớ không thay đổi), số đường dữ liệu là 8 đường và số chip sử dụng thiết kế là 2 ($k = 2$).



Thiết kế tăng số lượng ngăn nhớ

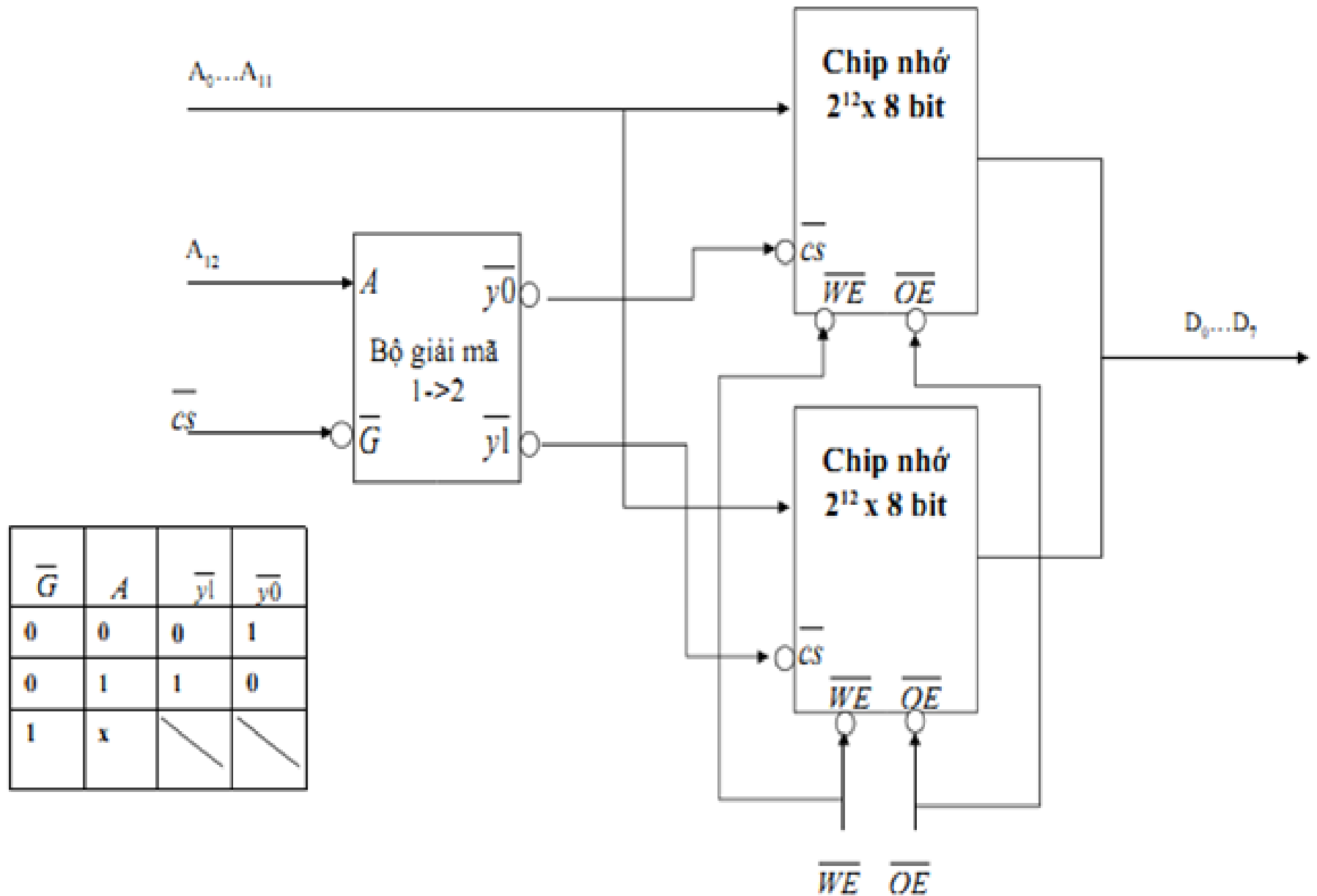
- Giả thiết: Cho các chip nhớ có dung lượng $2^n \times m$ bit.
- Yêu cầu: Thiết kế module nhớ có kích thước: $2^k \cdot 2^n \times m$ bit
- Giải quyết: Để thiết kế được ta xác định hai thông số $n + k$ (số đường địa chỉ) và 2^k (số chip nhớ cần để ghép vào module thiết kế).'

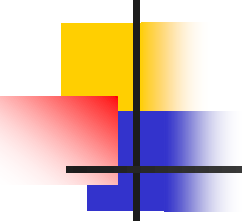


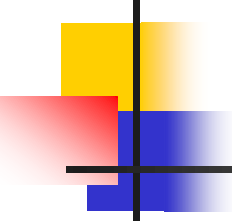
Ví dụ: Cho các chip nhớ SDRAM dung lượng 4K x 8 bit. Hãy thiết kế module nhớ có kích thước 8K x 8bit.

■ Ta có:

- Dung lượng chip nhớ giả thuyết $2^{12} \times 8$ bit
- Thông tin cần cho chip nhớ số đường địa chỉ $n = 12$ và số đường dữ liệu $m = 8$.
- Thông tin về module nhớ số đường địa chỉ là 13 đường (số ngăn nhớ thay đổi) và số đường dữ liệu là 8 đường (độ dài từ nhớ không đổi).



- 
-
- Bài tập
 - Thiết kế module nhớ 16Kx8bit từ các chip nhớ 4K x 8bit.
 - Thiết kế module nhớ 32Kx8bit từ các chip nhớ 4K x 8bit.
 - Thiết kế module nhớ 8Kx8bit từ các chip nhớ 4K x 4bit.
 - Thiết kế module nhớ 32Mx32bit từ các chip nhớ 4M x 32bit.



6.6. Phát hiện và chỉnh lỗi trong bộ nhớ.

